色紙認識の仕組み 解説

対象ファイル:AI生成試験版.py の色紙(R/G/B の単色の紙)に関係する部分
「どう動いているか」を中心に、コードに何が書いてあるかを順番に説明します

この部分がすること

床に置いた **赤・緑・青の単色の紙** を、ドローンのカメラ映像から探します。見つけたらその紙まで飛んで行き、真上で位置を合わせてから着陸します。流れはQRコードのときとまったく同じです。

QRコードとの一番大きな違いは、**「どこにあるか」を誰が計算しているか** です。

	QRコード	色紙
認識するところ	ドローン本体(pyhula の関数が結果を返す)	パソコン(このプログラムが画像を解析する)
使う関数	api.single_fly_Anticipatory_recognition	recognize_color(自分で作った関数)
返ってくる位置	カメラ基準 → camera_to_drone で変換が必要	最初からドローン基準(変換不要)
必要な情報	QRの番号だけ	色、カメラの角度、高度、カメラの視野角

つまり、QRのときはドローンが自動でやってくれていた「見つけて、位置を出す」部分を、色紙では **カメラ画像を受け取って自分で計算している** ということです。この資料の大部分は、その計算の説明です。

この資料の読み方

- 灰色の枠はプログラムのコードです。左の数字は AI生成試験版.py の行番号です。
- 青い枠「文法ポイント」は、コードに書いてある書き方の意味です。
- 黄色い枠「注意」は、実機で使うときに気をつける点です。
- Python の基本(変数、if、while、関数の書き方など)は、前に作った 1_改良版_解説.pdf で説明しています。この資料では、色紙の部分で新しく出てくるものを中心に説明します。

1. 全体の流れ

色紙の部分は、次の関数でできています。上の関数ほど「部品」、下の関数ほど「全体のまとめ役」です。

関数	役割	QRでいうと
pixel_to_floor	画面上の位置（ピクセル）を、床の上の位置（cm）に変換する	（ドローン本体が内部でやっている）
make_color_mask	指定した色の部分だけを白く塗った画像を作る	（同上）
recognize_color	カメラ画像から色紙を探し、位置を返す	single_fly_Anticipatory_recognition
color_get_closer	指定した距離だけ左右・前後に動く	qr_get_closer
color_auto_get_closer	認識して近づき、もう一度認識する	auto_get_closer
search_color	見つかるまで探す	search_qr
land_on_color	真上で位置を合わせながら降りて着地	land_on_qr
takeoff_and_land_on_color	離陸から着陸までの全部	takeoff_and_land_on_qr

色紙を1回認識するときの流れ（recognize_color の中身）

この資料の中心になる部分です。カメラ画像1枚から、色紙の位置を次の順番で求めます。

順番	すること	使うもの
1	少し待ってから、カメラ画像を1枚受け取る	api.get_image_array()
2	画像の色の形式を、色を判定しやすい HSV という形式に変換する	cv2.cvtColor
3	指定した色の部分だけを白、それ以外を黒にした画像（マスク）を作る	make_color_mask
4	白い部分の「かたまり」を探し、一番大きいものを色紙とみなす	cv2.findContours
5	かたまりが小さすぎたら「見つからない」とする（ノイズ対策）	color_min_area
6	かたまりの中心（重心）が、画面のどこにあるかを求める	cv2.moments
7	ドローンの高度を測る	api.get_plane_distance()
8	画面上の位置・カメラの角度・高度から、床の上の位置を計算する	pixel_to_floor

どこか1つの段階でもうまくいかなければ、その時点で {"result": False}（見つからない）を返します。

離陸から着陸までの流れ

こちらはQRのときと同じです。takeoff_and_land_on_color が全体をまとめています。

順番	すること	関数
1	離陸する	takeoff_and_land_on_color
2	色紙が見つかるまで、前進（または右回転）しながら探す	search_color
3	見つかったら、距離の9割だけ色紙に近づく	takeoff_and_land_on_color

順番	すること	関数
4	カメラを真下(90度)に向けて、20cm上がる	takeoff_and_land_on_color
5	真下を見ながら近づく。12cm未満なら着地、30cm未満なら35cm下がる	land_on_color
6	途中で見失ったら高さを調整して、2に戻る	land_on_color

2. 色紙用の設定値（20〜53行）

ファイルの上の方に、色紙の認識で使う数値がまとめてあります。実機に合わせて調整するのは、主にここです。

```
# ---------- 色紙の認識に使う設定値 ----------

# カメラの水平方向の視野角(度)
#   測り方:カメラを真下に向けて高さ h(cm) から床を映し、
#           画面の左端から右端までの長さ L(cm) を測る
#           視野角 = 2 × atan((L / 2) / h) を度に直した値
camera_hfov = 60

# カメラ画像の色の並び
#   pyhula の画像は RGB の順番なので True
#   実機で赤と青が入れ替わって認識される場合は False にする
image_is_rgb = True

# 画像を取得する前に待つ時間(秒)
color_wait_time = 0.5

# 色紙とみなす最小の大きさ(ピクセル数)
color_min_area = 500

# 認識する色の範囲(HSV)
#   [H(色相), S(鮮やかさ), V(明るさ)] の下限と上限
#   OpenCV では H は 0〜179、S と V は 0〜255
#   部屋の明るさや紙の色に合わせて調整すること
# 赤は H が 0 付近と 179 付近の両端にあるので、範囲を2つ使う
red_lower_1 = [0, 120, 70]
red_upper_1 = [10, 255, 255]
red_lower_2 = [170, 120, 70]
red_upper_2 = [179, 255, 255]
# 緑
green_lower = [40, 80, 50]
green_upper = [85, 255, 255]
# 青
blue_lower = [100, 120, 50]
blue_upper = [130, 255, 255]
```

変数	意味	調整の目安
`camera_hfov`	カメラの横方向の視野角(度)。**今の 60 は仮の値**	実際に測った値にする(測り方はコメントのとおり)。位置の計算の正確さに直接関係する
`image_is_rgb`	カメラ画像の色の並びが RGB かどうか	赤い紙を探して青に反応するなら `False` にする
`color_wait_time`	画像を取る前に待つ秒数	移動直後の古い映像を使ってしまうなら長くする
`color_min_area`	色紙とみなす最小の大きさ(ピクセル数)	関係ない物に反応するなら大きく、遠くの紙を見逃すなら小さく
`red_lower_1` など	色の範囲(HSV)。下限と上限	4章を参照

> **文法ポイント:リスト `[ , , ]`**
>
> `[0, 120, 70]` のように `[ ]` の中に値をカンマで並べたものを **リスト** といいます。複数の値をひとまとめにして1つの変数に入れられます。
>
> ここでは「H, S, V の3つの値の組」を表すために使っています。

> **文法ポイント:関数の外の変数を、関数の中で読む**
>
> `camera_hfov` などはファイルの一番外側(関数の外)で作った変数です。このような変数は、関数の中から **読むだけなら** そのまま使えます。
>
> 設定値をファイルの上にまとめておけば、関数の中を書き換えなくても、数字を変えるだけで動きを調整できます。

3. 先に知っておくこと:コンピューターの中の画像

画像は数字の表

カメラ画像は、コンピューターの中では **小さな点(ピクセル)が縦横に並んだ表** です。それぞれのピクセルが、色を表す3つの数字を持っています。

- `api.get_image_array()` で受け取る画像は、NumPy の配列(数字の表)です。
- `frame.shape` で大きさが分かります。例えば `(480, 640, 3)` なら、**縦480 × 横640 ピクセル、1ピクセルにつき色が3つ** という意味です。
- 位置は **左上が (0, 0)** で、右に行くほど x が、**下に行くほど** y が大きくなります。普段のグラフとは上下が逆なので注意してください。

RGB・BGR・HSV

形式	3つの数字の意味	このプログラムでの使い方
RGB	赤(R)・緑(G)・青(B)の強さ。0〜255	pyhula から受け取る画像の形式
BGR	RGB と同じだが、並び順が 青・緑・赤	OpenCV が普通に使う形式。変換の途中で使う
HSV	色相(H)・鮮やかさ(S)・明るさ(V)	色を判定するときに使う

なぜ HSV に変換するのか: RGB だと、同じ赤い紙でも、明るい場所と暗い場所で3つの数字が全部変わってしまいます。HSV なら「何色か」は H(色相)だけで決まり、明るさは V に分かれます。そのため「H がこの範囲で、ある程度鮮やかなら赤」のように、明るさが変わっても判定しやすくなります。

値	意味	OpenCV での範囲
H(色相)	何色か。赤 → 黄 → 緑 → 青 → 紫 → 赤 と一周する	0〜179(赤は 0 付近と 179 付近の両端)
S(彩度)	鮮やかさ。0 だと白・灰色・黒	0〜255
V(明度)	明るさ。0 だと真っ黒	0〜255

目安として、赤は H = 0〜10 と 170〜179、緑は 40〜85、青は 100〜130 あたりです。設定値もこの範囲になっています。

4. make_color_mask:指定した色だけを取り出す（301〜316行）

```
301 # HSV画像から、指定した色の部分だけを白、それ以外を黒にした画像(マスク)を作る関数
302 # color_name : "R"、"G"、"B" のどれか
303 # 返り値 : マスク画像(色の名前が間違っているときは None)
304 def make_color_mask(hsv, color_name):
305     if color_name == "R":
306         # 赤は範囲が2つあるので、それぞれのマスクを作って合わせる
307         mask_1 = cv2.inRange(hsv, np.array(red_lower_1), np.array(red_upper_1))
308         mask_2 = cv2.inRange(hsv, np.array(red_lower_2), np.array(red_upper_2))
309         return cv2.bitwise_or(mask_1, mask_2)
310     if color_name == "G":
311         return cv2.inRange(hsv, np.array(green_lower), np.array(green_upper))
312     if color_name == "B":
313         return cv2.inRange(hsv, np.array(blue_lower), np.array(blue_upper))
314 
315     print(f"色の指定が間違っています: {color_name}(\"R\", \"G\", \"B\" のどれかにする)")
316     return None
```

どう動いているか

HSV画像の全ピクセルを1つずつ調べ、**色の範囲に入っているピクセルを白(255)、入っていないピクセルを黒(0)** にした画像を作ります。この白黒の画像を **マスク** といいます。マスクの白い部分が「指定した色の部分」です。

- `cv2.inRange(画像, 下限, 上限)`:H・S・V の3つが **全部** 下限〜上限の範囲に入っていれば白にします。
- 赤だけは範囲が H の両端(0付近と179付近)に分かれているので、2つのマスクを作って `cv2.bitwise_or` で合わせます。`bitwise_or` は「どちらか片方でも白なら白」にする処理です。
- 緑と青は範囲が1つなので、`inRange` を1回呼ぶだけです。
- `"R" "G" "B"` 以外が指定されたら、メッセージを表示して `None` を返します。

例えば赤の1つ目の範囲 `[0, 120, 70]`〜`[10, 255, 255]` は、「色相が 0〜10(赤)で、鮮やかさが 120 以上、明るさが 70 以上」という意味です。S と V に下限があるのは、白っぽい色や暗い影を赤と間違えないようにするためです。

> **文法ポイント:np.array(リスト)**
>
> `np.array([0, 120, 70])` は、普通のリストを NumPy の配列に変換しています。OpenCV の関数は NumPy の配列を受け取る決まりなので、この変換が必要です。

> **文法ポイント:if の中の return で分岐する**
>
> 305〜313行は `if` が3つ並んでいて、それぞれの中で `return` しています。当てはまった `if` の中で関数が終わるので、`elif` を使わなくても「どれか1つだけ」が実行されます。どれにも当てはまらなかったときだけ315行に進みます。

> **文法ポイント:文字列の中の \"**
>
> 315行の `\"R\"` のように `\` をつけると、文字列の中に `"` を文字として入れられます。`\` がないと、そこで文字列が終わったと判断されてエラーになります。

5. recognize_color:色紙を認識する(319〜384行)

この部分の中心となる関数です。QRの `api.single_fly_Anticipatory_recognition` と同じ形の辞書を返すように作ってあります。長いので3つに分けて説明します。

5-1. 画像を受け取って、色の部分を取り出す(319〜345行)

```
319 # 色紙を認識する関数(QRの api.single_fly_Anticipatory_recognition の代わり)
320 # color_name   : "R"、"G"、"B" のどれか
321 # camera_angle : 今のカメラの角度
322 # 返り値は辞書
323 #   見つかった場合   : {"result": True, "x": x, "y": y, "z": z, "area": 面積}
324 #   見つからない場合 : {"result": False}
325 #   ※ QRと違い、x, y, z は最初から「ドローン基準」の位置(cm)
326 #      そのため camera_to_drone で変換する必要はない
327 def recognize_color(color_name, camera_angle):
328     # 映像が追いつくまで少し待ってから、カメラ画像を取得する
329     time.sleep(color_wait_time)
330     frame = api.get_image_array()
331     if frame is None:
332         print("カメラ画像が取得できませんでした")
333         return {"result": False}
334 
335     # OpenCV で扱えるように、色の並びを BGR にしてから HSV に変換する
336     if image_is_rgb:
337         bgr = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
338     else:
339         bgr = frame
340     hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
341 
342     # 指定した色の部分だけを取り出す
343     mask = make_color_mask(hsv, color_name)
344     if mask is None:
345         return {"result": False}
```

- 329行:`color_wait_time` 秒だけ待ちます。ドローンが動いた直後は、映像が少し遅れて届くからです。待たないと、動く前の古い画像で判断してしまう可能性があります。
- 330〜333行:カメラ画像を受け取ります。映像がまだ届いていないなどで取れなかったときは `None` が返るので、「見つからない」として終わります。
- 336〜340行:画像の形式を RGB → BGR → HSV と変換します。OpenCV の HSV 変換は BGR から行うのが基本なので、一度 BGR にしています。
- 343〜345行:4章の関数でマスクを作ります。色の指定が間違っていたら終わります。

> **文法ポイント:is None**
>
> `frame is None` は「frame が None(何もない)かどうか」を調べる書き方です。None かどうかを調べるときは `==` ではなく `is` を使うのが Python の決まりごとです。

5-2. 一番大きい色のかたまりを探して、中心を求める(347〜368行)

```
347    # 色のかたまり(輪郭)を探す
348    contours, hierarchy = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
349    if len(contours) == 0:
350        return {"result": False}
351
352    # 一番大きいかたまりを色紙とみなす
353    biggest = contours[0]
354    for contour in contours:
355        if cv2.contourArea(contour) > cv2.contourArea(biggest):
356            biggest = contour
357
358    # 小さすぎるかたまりはノイズとして無視する
359    area = cv2.contourArea(biggest)
360    if area < color_min_area:
361        return {"result": False}
362
363    # かたまりの中心(重心)の位置を求める
364    moments = cv2.moments(biggest)
365    if moments["m00"] == 0:
366        return {"result": False}
367    center_x = moments["m10"] / moments["m00"]
368    center_y = moments["m01"] / moments["m00"]
```

348行 `cv2.findContours`: マスクの白い部分の 輪郭(ふち) を探します。白い部分が離れて何か所かあれば、その数だけ輪郭が見つかります。結果は輪郭のリスト(`contours`)で返ってきます。

- `cv2.RETR_EXTERNAL`:一番外側の輪郭だけを探す、という指定です(穴の内側の輪郭は無視します)。
- `cv2.CHAIN_APPROX_SIMPLE`:輪郭の点を省略して記録する、という指定です(処理が軽くなります)。
- `hierarchy` は輪郭どうしの関係を表す情報ですが、このプログラムでは使っていません。受け取るためだけに書いています。

349〜350行: 輪郭が1つもない(指定した色が画面にない)なら、「見つからない」として終わります。

353〜356行: 見つかった輪郭の中から、**一番面積が大きいもの** を探して `biggest` に入れます。画面の中に小さな赤い物がいくつか映っていても、一番大きいもの = 色紙だと考えます。

359〜361行: 一番大きいかたまりでも `color_min_area` より小さければ、ノイズ(光の反射や小物)とみなして「見つからない」にします。

364〜368行: かたまりの **重心(中心)** を求めます。`cv2.moments` は図形の面積や重心を計算するための値をまとめて返します。

値	意味
`moments["m00"]`	面積
`moments["m10"]`	各点の x 座標を全部足したもの

値	意味
`moments["m01"]`	各点の y 座標を全部足したもの

「座標の合計 ÷ 面積」で座標の平均、つまり重心になります。面積が 0 だと割り算ができないので、365行で先に確認しています。

> **文法ポイント:for 文(繰り返し)**
>
> `for contour in contours:` は、`contours` に入っている輪郭を **1つずつ順番に取り出して** `contour` に入れ、中身を実行します。全部取り出し終わると、ループが終わります。
>
> `while` は「条件が正しい間」繰り返すのに対し、`for` は「リストの中身の数だけ」繰り返します。

```
for color in ["R", "G", "B"]:
    print(color)       # R, G, B と順番に表示される
```

> **文法ポイント:一番大きいものを探す定番の書き方**
>
> 353〜356行は「とりあえず最初の1つを一番大きいことにしておき、もっと大きいものが見つかったら入れ替える」という書き方です。最大値を探すときによく使う形なので、覚えておくと便利です。

> **文法ポイント:値を2つ受け取る**
>
> 348行の `contours, hierarchy = cv2.findContours(...)` は、関数が返す2つの値を2つの変数で受け取っています。自分で作った関数で `return x, y` と書いたときと同じ仕組みです。

5-3. 高度を測って、床の上の位置に変換する（370〜384行）

```
370    # ドローンの高度を取得する(ToFセンサー。単位は cm と思われる)
371    height = api.get_plane_distance()
372    if height is None or height <= 0:
373        print("高度が取得できませんでした")
374        return {"result": False}
375
376    # 画像の上の位置を、床の上の位置(ドローン基準)に直す
377    image_height = frame.shape[0]
378    image_width = frame.shape[1]
379    position = pixel_to_floor(center_x, center_y, image_width, image_height, camera_angle, height)
380    if position is None:
381        return {"result": False}
382
383    x, y = position
384    return {"result": True, "x": x, "y": y, "z": -height, "area": area}
```

- 371〜374行:ToFセンサーでドローンの高度を測ります。取れなかったときや 0 以下のときは、位置を計算できないので「見つからない」にします。
- 377〜378行:`frame.shape` から画像の縦と横の大きさを取り出します。`shape[0]` が縦、`shape[1]` が横です(順番に注意)。
- 379〜381行:6章の `pixel_to_floor` で、画面上の重心の位置を床の上の位置(cm)に変換します。地平線より上で床に届かないときは `None` が返るので、「見つからない」にします。
- 383〜384行:結果を辞書にして返します。`z` は「床はドローンより高度の分だけ下にある」ので `-height`(マイナス)です。`area` は、どのくらいの大きさで映ったかを確認するためのおまけです。

文法ポイント:A or B

`if height is None or height <= 0:` は「height が None、**または** 0 以下なら」という意味です。

`or` は左から順に調べて、左側が正しければ右側は調べません。そのため height が None のときに `None <= 0`(エラーになる比較)は実行されません。**順番に意味があります。**

文法ポイント:[0] [1] で取り出す

`frame.shape[0]` のように `[番号]` をつけると、並んでいる値の何番目かを取り出せます。**番号は 0 から始まります。**`(480, 640, 3)` なら `[0]` が 480、`[1]` が 640、`[2]` が 3 です。

文法ポイント:x, y = position

`position` には `(x, y)` の2つの値の組が入っています。`x, y = position` で、それを2つの変数に分けて取り出しています。

注意:高度の単位

`get_plane_distance()` の単位は、pyhula の説明に書かれていません。cm だと考えて計算しています。もし mm などで返ってくるなら、位置の計算結果がその分だけずれます。実機で確認してください。

6. pixel_to_floor:画面の位置から床の位置を求める(268～298行)

色紙の位置を出すための計算の中心です。三角関数を使いますが、考え方は単純です。

```
268 # 画像の上の位置(ピクセル)から、床の上の位置(ドローン基準, cm)を求める関数
269 # 床が平らだと考えて、「カメラから物体へ向かう線」が床とぶつかる点を計算する
270 # pixel_x, pixel_y : 物体の中心の位置(画像の左上が 0, 0)
271 # image_width, image_height : 画像の横幅と縦幅(ピクセル)
272 # camera_angle : カメラの角度(水平から下向きに何度か)
273 # height       : ドローンの高度(cm)
274 # 返り値 : (x, y)  x は右が正、y は前が正(cm)
275 #          床に届かない向き(地平線より上)のときは None
276 def pixel_to_floor(pixel_x, pixel_y, image_width, image_height, camera_angle, height):
277     # 視野角から、レンズの焦点距離をピクセル単位で求める
278     focal = (image_width / 2) / math.tan(math.radians(camera_hfov) / 2)
279 
280     # 画面の中心からのずれを「向き」に直す
281     #   right_ratio : 右にどれだけずれているか(画面の右が正)
282     #   up_ratio    : 上にどれだけずれているか(画面の上が正)
283     right_ratio = (pixel_x - image_width / 2) / focal
284     up_ratio = (image_height / 2 - pixel_y) / focal
285 
286     # カメラの傾きを考えて、線がどれだけ下に向かっているかを求める
287     angle_rad = math.radians(camera_angle)
288     down_ratio = math.sin(angle_rad) - up_ratio * math.cos(angle_rad)
289 
290     # 下に向かっていない線は床とぶつからない
291     if down_ratio <= 0:
292         return None
293 
294     # 床とぶつかる点を求める
295     distance = height / down_ratio
296     x = right_ratio * distance
297     y = (math.cos(angle_rad) + up_ratio * math.sin(angle_rad)) * distance
298     return x, y
```

考え方

カメラに映っている点は、どれも **カメラから一直線に伸びた線の先** にあります。画面の中心に映っている物はカメラの正面方向に、画面の右の方に映っている物はカメラから右寄りの方向にあります。

床が平らなら、その線をまっすぐ伸ばして **床とぶつかった点** が物体の位置です。ぶつかる点を求めるのに必要なのは、次の3つです。

- **線の向き**:画面上の位置と、カメラの視野角から分かる
- **カメラの傾き**:`camera_angle`
- **床までの高さ**:`height`

計算の手順

行	計算	意味
278	`focal`	視野角から求めた「レンズの焦点距離」をピクセル単位で表したもの。画面の中心から端までのピクセル数と、視野角の半分の tan の比で決まる

行	計算	意味
283	`right_ratio`	画面の中心から右へのずれ ÷ focal。「正面に1進む間に、右にどれだけずれるか」を表す線の傾き
284	`up_ratio`	同じく上へのずれ。画像の y は下向きが正なので、`中心 - pixel_y` と逆にして「上が正」にしている
288	`down_ratio`	カメラの傾きを考えて、線が「下へどれだけの割合で向かっているか」
291	`down_ratio <= 0`	下に向かっていない（地平線かそれより上）なら、床とぶつからないので None を返す
295	`distance`	高さ ÷ 下向きの割合。線が床に届くまでに進む量
296〜297	`x, y`	床に届いた点の、右方向（x）と前方向（y）の距離

計算例

画像が 横640 × 縦480 ピクセル、視野角 60度、高度 100cm の場合で計算してみます。このとき `focal` は 554.3 になります。

例1:カメラ真下（90度）、紙の中心が (480, 240) に映った場合

- right_ratio = (480 - 320) ÷ 554.3 = 0.289、up_ratio = (240 - 240) ÷ 554.3 = 0
- down_ratio = sin90° - 0 × cos90° = 1、distance = 100 ÷ 1 = 100
- x = 0.289 × 100 = **28.9 cm（右）**、y = (cos90° + 0) × 100 = **0.0 cm**

→ 紙はドローンの真下から右に約29cmの位置にある、という結果になります。

例2:カメラ45度、紙の中心が (320, 120) に映った場合（画面の真ん中より上）

- right_ratio = 0、up_ratio = (240 - 120) ÷ 554.3 = 0.217
- down_ratio = sin45° - 0.217 × cos45° = 0.554、distance = 100 ÷ 0.554 = 180.5
- x = **0.0 cm**、y = (cos45° + 0.217 × sin45°) × 180.5 = **155.3 cm（前）**

→ 斜め前を見ているので、画面の上の方に映った物は、かなり前方（約1.5m先）にあることになります。

> **文法ポイント:math の関数**
>
> `math.radians(角度)`:度をラジアンに直す。`math.sin` や `math.cos`、`math.tan` はラジアンでしか計算できないため。
>
> `math.tan(…)`:タンジェント。視野角から焦点距離を求めるのに使っている。

> **注意:計算の前提と誤差**
>
> 床が平らで、ドローンが水平だと考えて計算しています。実際には飛行中に機体が少し傾くので、その分ずれます。
>
> カメラの傾きが水平に近いほど（角度が小さいほど）、少しのずれで遠くの位置が大きく変わります。**真下に近い角度ほど正確** です。
>
> カメラが機体の中心からずれた位置に付いている場合、そのずれは計算に入っていません。
>
> `camera_hfov`（視野角）が間違っていると、全体の距離がその割合でずれます。

7. color_get_closer と color_auto_get_closer:近づく(387〜440行)

color_get_closer(387〜401行)

```
387 # 色紙に近づく関数(QRの qr_get_closer と同じ動き)
388 # relative_x, relative_y : ドローン基準での色紙の相対位置(cm)
389 # scale                  : 移動距離にかける倍率(1なら相対位置の分だけそのまま動く)
390 def color_get_closer(relative_x, relative_y, scale):
391     # 左右の移動
392     if relative_x > 0:
393         api.single_fly_right(int(relative_x * scale), moving_speed)
394     if relative_x < 0:
395         api.single_fly_left(int(abs(relative_x) * scale), moving_speed)
396 
397     # 前後の移動
398     if relative_y > 0:
399         api.single_fly_forward(int(relative_y * scale), moving_speed)
400     if relative_y < 0:
401         api.single_fly_back(int(abs(relative_y) * scale), moving_speed)
```

受け取った位置の方向へ、左右と前後に動くだけの関数です。中身はQRの `qr_get_closer` とまったく同じです。QRと色紙の処理を分けるために、別の名前で用意しています。

color_auto_get_closer(404〜440行)

```
404 # 色紙を認識してから近づく関数(QRの auto_get_closer と同じ動き)
405 # 返り値は辞書で、色紙の相対位置は「移動後」に認識し直したもの
406 #   見つかった場合   : {"result": True, "relative_x": x, "relative_y": y, "relative_z": z}
407 #   見つからない場合 : {"result": False}
408 def color_auto_get_closer(scale, color_name, camera_angle):
409     array = recognize_color(color_name, camera_angle)
410 
411     # 色紙が見つからなかった場合
412     if not array["result"]:
413         print("COLOR NOT FOUND")
414         return {"result": False}
415 
416     relative_x = array["x"]
417     relative_y = array["y"]
418     relative_z = array["z"]
419 
420     # 既に十分近い(左右前後とも10cm未満)なら動かない
421     if abs(relative_x) < 10 and abs(relative_y) < 10:
422         print("既に十分近い位置にいます")
423         return {"result": True,
424                 "relative_x": relative_x,
425                 "relative_y": relative_y,
426                 "relative_z": relative_z}
427 
428     # 近づいてから、もう一度認識する
429     print(f"ドローン基準で色紙の相対位置({relative_x:.2f},{relative_y:.2f},{relative_z:.2f})に近づきます")
430     color_get_closer(relative_x, relative_y, scale)
431     array = recognize_color(color_name, camera_angle)
432 
```

```
433     # 近づいたあとに見失った場合
434     if not array["result"]:
435         return {"result": False}
436 
437     return {"result": True,
438             "relative_x": array["x"],
439             "relative_y": array["y"],
440             "relative_z": array["z"]}
```

色紙を認識して近づき、もう一度認識して、近づいたあとの位置を返します。land_on_color の中で繰り返し使われます。

- 409〜414行:認識して、見つからなければ {"result": False} を返します。
- 416〜418行:位置を取り出します。**QRと違って camera_to_drone による変換はしません**(最初からドローン基準の位置なので)。
- 421〜426行:左右・前後とも10cm未満なら、動かずにそのまま返します。
- 429〜431行:近づいてから、もう一度認識します。
- 434〜435行:近づいたあとに見失ったら {"result": False} を返します。

> **文法ポイント:QR版との違い(434〜435行)**
>
> QR版の auto_get_closer は、2回目の読み取りで見失っても位置を使って計算します(pyhula が位置に意味のない値を入れて返すため)。
>
> 色紙版の recognize_color は、見つからないときは位置を返さないので、ここで先に確認しています。確認しないと array["x"] がなくてエラーになります。

8. 探す・降りる・全体のまとめ(443〜570行)

この3つの関数は、QRの search_qr、land_on_qr、takeoff_and_land_on_qr と同じ流れです。違うのは「認識に recognize_color を使う」「位置の変換をしない」「表示するメッセージ」の3点だけです。

search_color:色紙を探す(443〜492行)

```
452 def search_color(color_name, search_mode, color_found_before, camera_angle, turn_angle):
453     while True:
454 
455         # 色紙が見つかった場合
456         array = recognize_color(color_name, camera_angle)
457         if array["result"]:
458             print(f"色紙({color_name})発見")
459             color_found_before = True
460             return array, color_found_before, camera_angle, turn_angle
461 
462         if color_found_before:
463             # 一度発見したのに見失った場合は最大2回まで下がる
464             found_again = False
465             back_count = 0
466             while back_count < 2:
467                 back_count += 1
468                 print("見失ったから後進")
469                 api.single_fly_back(40)
470                 camera_angle = set_camera_angle(60)
```

```
                array = recognize_color(color_name, camera_angle)
                if array["result"]:
                    print("再発見")
                    found_again = True
                    break

            if found_again:
                return array, color_found_before, camera_angle, turn_angle

            # 2回下がっても見つからなかったら、一度も見つけていない状態に戻す
            print("後進したが見つからないのでリセット")
            color_found_before = False

        else:
            # まだ一度も見つけていなくて発見できない場合
            if search_mode == "forward":
                api.single_fly_forward(25)
                print("色紙ないから前進")
            if search_mode == "turn":
                api.single_fly_turnright(30)
                turn_angle += 30
                print(f"色紙ないから右30度回転(累積{turn_angle}度)")
```

- 456～460行:見つかれば結果を返して終わります。
- 462～482行:一度見つけたのに見失ったときは、40cm後ろに下がってカメラを60度にし、もう一度探します。これを最大2回繰り返します。
- 484～492行:一度も見つけていないときは、前進("forward")か右回転("turn")をして、また探します。回転したときは、回った角度を `turn_angle` に足していきます。

land_on_color:真上から降りて着地する(495～533行)

```
def land_on_color(color_name, camera_angle, turn_angle):
    # 高度を下げた直後かどうか
    low_height = False

    while True:
        auto_closer_return = color_auto_get_closer(1, color_name, camera_angle)
        print(auto_closer_return)

        if auto_closer_return["result"]:

            # 左右前後とも12cm未満なら真上にいるとみなして着地
            if abs(auto_closer_return["relative_x"]) < 12 and abs(auto_closer_return["relative_y"]) < 12:
                # 斜めのまま着地しないよう、累積角度を90で割った余りだけ左に戻す
                remainder = turn_angle % 90
                if remainder != 0:
                    print(f"向きを揃えるため左に{remainder}度回転")
                    api.single_fly_turnleft(remainder)
                print("真上なので着地します")
                api.single_fly_touchdown()
                return True

            # 左右前後とも30cm未満なら高度を下げる
            if abs(auto_closer_return["relative_x"]) < 30 and abs(auto_closer_return["relative_y"]) < 30:
                print("近いので高度を下げます")
                api.single_fly_down(35)
```

```
523                 low_height = True
524
525         else:
526             # 見失った場合は高度を調整してから探し直す
527             print("見逃した")
528             if low_height:
529                 low_height = False
530                 api.single_fly_up(15)
531             else:
532                 api.single_fly_down(20)
533             return False
```

- 503行:7章の color_auto_get_closer で近づきます。
- 509～517行:左右・前後とも12cm未満なら、真上にいるとみなします。回転した角度を % 90(90で割った余り)で求め、その分だけ左に戻して向きをそろえてから着地します。
- 520～523行:30cm未満なら35cm下がります。
- 525～533行:見失ったら、直前に下がっていれば15cm上がり、そうでなければ20cm下がって、False を返します。

> **文法ポイント:% (割り算の余り)**
>
> turn_angle % 90 は「turn_angle を 90 で割った余り」です。例えば 120 % 90 = 30 なので、30度左に戻せば、90度の倍数の向きにそろいます。

takeoff_and_land_on_color:離陸から着陸まで(536～570行)

```
541 def takeoff_and_land_on_color(color_name, search_mode, camera_angle):
542     color_found_before = False
543     # 探索中に右へ回転した累積角度
544     turn_angle = 0
545
546     # 離陸
547     api.single_fly_takeoff()
548
549     # 着地できるまで「探す → 近づく → 降りる」を繰り返す
550     while True:
551         # 色紙を探す
552         array, color_found_before, camera_angle, turn_angle = search_color(color_name, search_mode, color_
found_before, camera_angle, turn_angle)
553
554         # 色紙に向かって行って下を向く
555         # (色紙の位置は最初からドローン基準なので、そのまま使う)
556         relative_x = array["x"]
557         relative_y = array["y"]
558         relative_z = array["z"]
559         print(f"ドローン基準での色紙の相対位置(x,y,z): {relative_x:.2f}, {relative_y:.2f}, {relative_z:.2f
}")
560         color_get_closer(relative_x, relative_y, 0.9)
561         print("接近したから真下を向く")
562         camera_angle = set_camera_angle(90)
563         time.sleep(1)
564         api.single_fly_up(20)
565
566         # 真上から近づいて着地する(見失ったらもう一度探すところから)
```

```
        landed = land_on_color(color_name, camera_angle, turn_angle)
        if landed:
            print(f"{color_name}の色紙に着陸しました")
            return
```

- 552行:色紙を探します。
- 556～558行:位置をそのまま取り出します(QR版ではここで `camera_to_drone` を使っています)。
- 560～564行:距離の9割だけ近づき、カメラを真下に向けて、1秒待ってから20cm上がります。
- 567～570行:真上から着地を試します。見失ったら 550行の `while True:` の先頭に戻って、探すところからやり直します。

> **注意:色の指定は 1つだけ**
>
> QRのように「探すときの目印」と「着陸する紙」を分ける機能は、色紙の方にはありません。探すときも降りるときも、同じ色を使います。

9. 使い方

```
# ---------- 色紙に着陸する場合の例 ----------
# 使うときは、行の先頭の「# 」を消す
# 色は "R"(赤)、"G"(緑)、"B"(青) で指定する
#
# time.sleep(10)
# camera_angle = set_camera_angle(45)
# takeoff_and_land_on_color("R", "turn", camera_angle)
```

メイン処理の最後に、使い方の例がコメントとして書いてあります。行の先頭の `#` を消すと実行されるようになります。

```
time.sleep(10)
camera_angle = set_camera_angle(45)
takeoff_and_land_on_color("R", "turn", camera_angle)
```

- 1つ目の値:着陸する色紙の色。"R"(赤)、"G"(緑)、"B"(青)のどれか
- 2つ目の値:見つからないときの探し方。"forward"(25cm前進)か "turn"(右に30度回転)
- 3つ目の値:今のカメラの角度

今のメイン処理は、QRの0番・1番・2番に着陸したあとにこの例が来る順番になっています。色紙だけを試したいときは、QRの部分を `#` でコメントにしてください。

10. 実機で試す前に確認すること

確認すること	確認のしかた	関係する設定
画像が取れるか	`print(api.get_image_array().shape)` で大きさが表示されるか	—
色の並び	赤い紙を "R" で認識できるか。青に反応するなら逆になっている	`image_is_rgb`
高度の単位	`print(api.get_plane_distance())` を、実際の高さ(cm)と比べる	—
視野角	コメントにある方法で測る	`camera_hfov`
位置の正しさ	紙を置いた場所をメジャーで測り、`print(recognize_color("R", 90))` の結果と比べる	`camera_hfov`
色の範囲	紙以外の物に反応しないか、部屋の明るさを変えても認識できるか	`red_lower_1` など
見つけられる距離	紙からどのくらい離れても見つけられるか	`color_min_area`

> **注意:見つけられる距離には限りがある**
>
> 遠くの紙は画面に小さく映るので、`color_min_area` より小さくなると「見つからない」になります。"turn" で探すと、その場で回るだけで前には進まないので、遠くの紙にはいつまでも近づけません(QRの場合も同じです)。
>
> 見つからないとずっと回り続けるので、実際に飛ばすときは、すぐに止められるようにしておいてください。

> **注意:一番大きい「その色の物」を紙だと考える**
>
> 画面の中に、紙より大きな同じ色の物（赤い服や箱など）が映ると、そちらを紙だと思って近づいてしまいます。
> 周りに同じ色の物を置かないようにしてください。